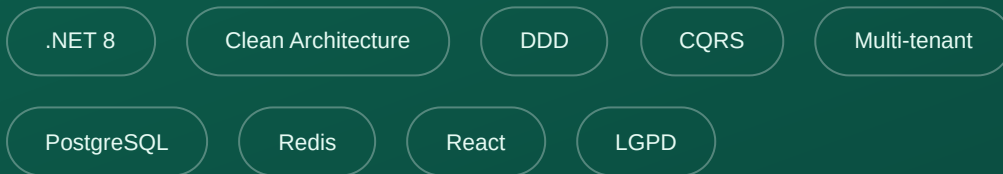


Plataforma de Gestão Odontológica

Arquitetura de referência escalável — monólito modular em .NET 8, Clean Architecture, DDD e CQRS, com caminho evolutivo para serviços independentes.



PADRÃO ARQUITETURAL

Monólito modular

VERSÃO

1.0

ESCOPO

Backend, frontend, dados e infra

Sumário

01	Visão geral e princípios	objetivos, drivers de qualidade, decisões-chave
02	Estilo arquitetural	monólito modular + Clean Architecture
03	Stack tecnológica	backend, frontend, dados, infraestrutura
04	Módulos de domínio	bounded contexts e responsabilidades
05	Design patterns por camada	táticos, aplicação e integração
06	Pacotes e bibliotecas	NuGet e npm com propósito
07	Multi-tenancy	isolamento de dados por clínica
08	Segurança e LGPD	dados clínicos, auditoria, autorização
09	Observabilidade	logs, métricas, tracing
10	Escalabilidade e evolução	do monólito ao serviço extraído
11	Estrutura de solução	organização de projetos e pastas
12	DevOps e CI/CD	build, testes, deploy, ambientes
13	Roadmap de implementação	fases e ordem de entrega

01 Visão geral e princípios

Uma plataforma que centraliza a operação de clínicas odontológicas — agendamento, prontuário eletrônico, financeiro e relacionamento com pacientes — desenhada desde o início para atender de uma clínica única a uma rede de unidades, sem reescrita de domínio ao crescer.

Drivers de qualidade priorizados

Atributo	Como a arquitetura responde
Escalabilidade	Fronteiras de módulo isoladas permitem extrair o módulo mais carregado (Agenda) como serviço próprio; cache de leitura em Redis absorve o tráfego de calendário.
Manutenibilidade	Clean Architecture + DDD mantêm o domínio livre de detalhes de infraestrutura; cada módulo evolui de forma independente.
Segurança / conformidade	Dados clínicos são categoria sensível sob a LGPD: criptografia em repouso, trilha de auditoria imutável e autorização por papel e por tenant.
Disponibilidade	Processamento assíncrono via fila desacopla tarefas não críticas (lembretes, faturas) do ciclo de requisição.
Testabilidade	CQRS + inversão de dependência tornam casos de uso testáveis sem banco; regras de domínio testadas em isolamento.

Decisões arquiteturais fundamentais

● Monólito modular, não microsserviços

Um único deploy com fronteiras de domínio rígidas. Entrega 90% do benefício organizacional dos microsserviços sem o custo operacional de um sistema distribuído.

● Multi-tenant desde o dia zero

Modelar `tenant_id` na base é barato agora e caríssimo de refatorar depois — mesmo que o MVP atenda uma só clínica.

● Comunicação entre módulos por contrato

Nenhum módulo faz `JOIN` ou acessa repositório de outro. A integração acontece por eventos de domínio e interfaces públicas.

● Escrita e leitura separadas (CQRS)

Comandos passam pelo domínio; consultas usam projeções otimizadas. Essencial no calendário, com leitura muito superior à escrita.

02 Estilo arquitetural

O sistema é um **monólito modular**: processo e deploy únicos, mas subdividido em módulos de domínio autônomos. Cada módulo é internamente estruturado em **Clean Architecture**, com o fluxo de dependências sempre apontando para dentro, em direção ao domínio.

As quatro camadas de cada módulo

Domain	Entidades, agregados, objetos de valor, eventos de domínio e regras de negócio puras. Zero dependência de framework.
--------	--

Application	Casos de uso (commands / queries), handlers, validações, orquestração. Define interfaces (portas) que a infraestrutura implementa.
Infrastructure	Implementações concretas: EF Core, repositórios, acesso a Redis, publicação em fila, integrações externas.
API / Host	Endpoints HTTP, autenticação, composição de dependências (DI), middleware transversal. Ponto de entrada único.

REGRA DE DEPENDÊNCIA

A seta de dependência aponta sempre para o centro: `API → Application → Domain`. A camada `Infrastructure` depende de `Application` e `Domain`, mas o inverso nunca ocorre — o domínio conhece apenas interfaces, jamais implementações concretas. Isso é o **Dependency Inversion Principle** aplicado na fronteira mais importante do sistema.

Módulo simples vs. monólito modular

Dimensão	Monólito acoplado	Monólito modular (adotado)
Fronteiras	Difusas; qualquer classe chama qualquer outra	Rígidas; comunicação só por contrato público
Acesso a dados	<code>JOIN</code> livre entre tabelas de domínios distintos	Cada módulo dono do seu schema; sem <code>JOIN</code> cruzado
Evolução	Extrair um serviço exige reescrever o domínio	Fronteira já existe; extração é trabalho de infra
Risco	Vira "big ball of mud" com o crescimento	Acoplamento controlado por design

03 Stack tecnológica

Backend

Tecnologia	Papel
.NET 8 / C# 12	Runtime LTS. Base de toda a aplicação server-side.
ASP.NET Core	Web framework — Minimal APIs ou Controllers para expor os endpoints.
Entity Framework Core 8	ORM principal; migrations, mapeamento de agregados, concorrência otimista.
PostgreSQL 16	Banco relacional primário. Suporte a JSONB (anexos de prontuário), particionamento e schema-per-tenant.
Redis	Cache distribuído (disponibilidade de agenda), locks distribuídos, sessão e rate limiting.
RabbitMQ	Broker de mensagens para processamento assíncrono e comunicação por eventos entre módulos.

Frontend

Tecnologia	Papel
React 18 + TypeScript	SPA para recepção, dentistas e administração. TypeScript garante contratos tipados com a API.
Vite	Build e dev server rápidos.
TanStack Query	Cache de estado de servidor, revalidação e sincronização — ideal para o calendário.
React Hook Form + Zod	Formulários performáticos com validação por schema, espelhando as regras do backend.
Tailwind CSS + shadcn/ui	Design system consistente, acessível e de rápida composição.
FullCalendar	Componente de calendário para a agenda (visões dia/semana, drag-and-drop de horários).

Infraestrutura e plataforma

Tecnologia	Papel
Docker + Docker Compose	Empacotamento e ambiente local reproduzível.
Kubernetes (opcional)	Orquestração quando houver necessidade de escala horizontal e alta disponibilidade.
NGINX / YARP	Reverse proxy, terminação TLS e balanceamento.
OpenTelemetry	Instrumentação padronizada de traces, métricas e logs.
GitHub Actions	Pipeline de CI/CD.

04 Módulos de domínio

Cada módulo é um **bounded context** com linguagem ubíqua e modelo próprios. Abaixo, as responsabilidades e os pontos de atenção de cada um.

● Agenda

MÓDULO DE MAIOR TRÁFEGO

Horários de profissionais, cadeiras/salas, marcação, remarcação e prevenção de conflitos.

- Concorrência: **RowVersion** (otimista) + lock curto no Redis ao confirmar
- Leitura de disponibilidade servida por cache
- Publica **ConsultaConcluidaEvent**

● Prontuário eletrônico

DADO CLÍNICO SENSÍVEL — LGPD

Histórico clínico, odontograma, evolução de tratamento e anexos (raio-x, exames).

- Criptografia em repouso dos campos clínicos
- Auditoria de todo acesso de leitura/escrita
- Anexos em object storage; metadados em JSONB

● Financeiro

INTEGRAÇÃO COMPLEXA

Faturamento particular e por convênio, parcelamento, boletos e comissão de dentistas.

- Convênios atrás de **camada anticorrupção**
- Geração de faturas via fila (assíncrono)
- Escuta eventos da Agenda para faturar

● Pacientes / CRM

RELACIONAMENTO

Cadastro, dados de contato, campanhas e lembretes de consulta.

- Notificações (SMS/WhatsApp/e-mail) via fila
- Consentimento e portabilidade de dados
- Base para comunicação e retorno

Módulos transversais de suporte

Módulo	Responsabilidade
Identity & Access	Autenticação, papéis (dentista, recepção, admin), autorização por tenant.
Tenancy	Resolução do tenant na requisição, gestão de clínicas/unidades de uma rede.
Estoque	Controle de materiais e insumos odontológicos (fase posterior).
Notifications	Serviço compartilhado de envio, consumido pelos demais módulos via eventos.

05 Design patterns por camada

Padrões escolhidos por problema concreto que resolvem — não por catálogo. Cada um abaixo tem um motivo claro dentro deste sistema.

Domínio (padrões táticos de DDD)

- Aggregate
- Entity
- Value Object
- Domain Event
- Domain Service
- Specification

Padrão	Aplicação neste sistema
Aggregate	Agendamento é raiz de agregado; garante invariantes (não sobrepor horário) em uma única fronteira transacional.
Value Object	PeriodoHorario , CPF , Dinheiro — imutáveis, comparados por valor, sem identidade própria.
Domain Event	ConsultaConcluida desacopla Agenda de Financeiro sem chamada direta.
Specification	Regras de elegibilidade (ex.: paciente apto a agendar) encapsuladas e reutilizáveis.

Aplicação

- CQRS
- Mediator
- Command / Query Handler
- Pipeline Behavior
- Result

Padrão	Aplicação neste sistema
CQRS	Comandos alteram estado pelo domínio; queries leem projeções otimizadas, sem carregar agregados completos.
Mediator	Desacopla o emissor da requisição do handler; centraliza cross-cutting concerns.
Pipeline Behavior	Validação, logging, transação e cache aplicados como middleware do fluxo de comandos.
Result pattern	Erros de negócio retornados como valor, não exceção — fluxo explícito e testável.

Infraestrutura e integração

- Repository
- Unit of Work
- Outbox
- Anti-Corruption Layer
- Options
- Decorator

Padrão	Aplicação neste sistema
Repository + UoW	Abstrai persistência; o DbContext do EF Core já atua como Unit of Work coeso por transação.
Outbox	Eventos gravados na mesma transação do dado; um worker os publica na fila — garante consistência sem perder mensagem.
Anti-Corruption Layer	Cada convênio tem seu adaptador; o formato externo nunca vaza para o domínio Financeiro.
Decorator	Cache e retry envolvem serviços sem alterar sua implementação.

06 Pacotes e bibliotecas

Seleção enxuta e madura. Cada dependência resolve um problema específico e é amplamente adotada no ecossistema.

NuGet — backend (.NET)

Pacote	Para quê
MediatR	Implementação de Mediator/CQRS e pipeline de comandos e queries.
FluentValidation	Validação declarativa de comandos, integrada ao pipeline do MediatR.
Npgsql.EntityFrameworkCore.PostgreSQL	Provider EF Core para PostgreSQL.
Mapster	Mapeamento objeto-objeto (entidade ↔ DTO) rápido e sem configuração pesada.
MassTransit	Abstração de mensageria sobre RabbitMQ; consumidores, retry e outbox integrados.
StackExchange.Redis	Cliente Redis para cache e locks distribuídos.
Serilog	Logging estruturado com sinks para console, arquivo e coletor central.
OpenTelemetry.*	Traces e métricas exportados por padrão aberto.
Microsoft.AspNetCore.Authentication.JwtBearer	Autenticação via JWT.
Quartz.NET	Agendamento de jobs recorrentes (lembretes, fechamento financeiro).
Polly	Resiliência: retry, circuit breaker e timeout em integrações externas.
Swashbuckle / NSwag	Documentação OpenAPI da API.

Testes

Pacote	Para quê
xUnit	Framework de testes unitários e de integração.
FluentAssertions	Asserções legíveis e expressivas.
Testcontainers	PostgreSQL e Redis reais em contêiner durante os testes de integração.
NSubstitute	Test doubles para isolar dependências.
Bogus	Geração de dados de teste realistas.

npm — frontend (React)

Pacote	Para quê
@tanstack/react-query	Estado de servidor, cache e revalidação.
react-hook-form + zod	Formulários e validação por schema.
axios ou ky	Cliente HTTP com interceptors (token, tenant).
@fullcalendar/react	Calendário da agenda.
tailwindcss + shadcn/ui	Estilo e componentes de UI.
zustand	Estado de cliente leve (UI, filtros, sessão).
vitest + @testing-library/react	Testes de componente e unidade.

07 Multi-tenancy

A plataforma isola dados por clínica desde a modelagem. A escolha da estratégia equilibra simplicidade operacional e grau de isolamento.

Estratégias e quando usar cada uma

Estratégia	Descrição	Recomendação
Shared schema (tenant_id)	Uma tabela por entidade, discriminada por coluna de tenant. Filtro global aplicado automaticamente.	Ponto de partida do MVP — simples e econômico.
Schema per tenant	Um schema PostgreSQL por clínica, mesmo banco. Isolamento lógico mais forte.	Ao crescer a rede e surgirem exigências de isolamento.
Database per tenant	Banco dedicado por cliente. Isolamento máximo, custo operacional alto.	Apenas para clientes enterprise específicos.

IMPLEMENTAÇÃO RECOMENDADA

Começar com **shared schema**. Um middleware resolve o `tenant_id` a partir do token/subdomínio no início da requisição e o injeta num `ITenantContext` . Um **global query filter** do EF Core (`HasQueryFilter`) aplica `WHERE tenant_id = @atual` em toda consulta, tornando o vazamento entre tenants um erro difícil de cometer por esquecimento.

Hierarquia de tenancy para redes

Para uma rede de clínicas, o modelo é hierárquico: **Tenant (empresa) → Unidade (clínica) → Recurso** . Um administrador da rede enxerga todas as unidades; um recepcionista, apenas a sua. Esse mesmo padrão hierárquico multi-nível já aparece em sistemas de gestão com filiais e se aplica diretamente aqui.

08 Segurança e conformidade (LGPD)

Prontuário odontológico é dado pessoal sensível de saúde. A conformidade não é um recurso opcional — é requisito estrutural.

Controle	Implementação
Autenticação	JWT de curta duração + refresh token; senhas com hash forte (Argon2/BCrypt).
Autorização	RBAC por papel e escopo de tenant; políticas do ASP.NET Core aplicadas por endpoint.
Criptografia em repouso	Campos clínicos sensíveis cifrados na aplicação; volume do banco cifrado no storage.
Criptografia em trânsito	TLS obrigatório em toda comunicação externa e entre serviços.
Trilha de auditoria	Log imutável de quem acessou qual prontuário, quando e o quê — append-only.
Direitos do titular	Consentimento explícito, portabilidade e exclusão dos dados sob solicitação.
Segregação por tenant	Filtro global impede que uma clínica acesse dados de outra.

PONTO CRÍTICO

A trilha de auditoria de prontuário deve ser **append-only** e independente do fluxo normal de dados — nem um administrador deve conseguir apagá-la sem deixar rastro. Grave os eventos de auditoria via Outbox para uma tabela dedicada ou store externo, garantindo que a auditoria persista mesmo se a operação principal falhar.

09 Observabilidade

Os três pilares instrumentados desde o início, sob o padrão OpenTelemetry, para diagnóstico rápido em produção.

● Logs

Serilog com logging estruturado. Cada log carrega **TraceId** , **TenantId** e **UserId** para correlação.

● Métricas

Contadores e histogramas exportados via OpenTelemetry (latência por endpoint, taxa de erro, tamanho de fila).

● Tracing distribuído

Rastro de ponta a ponta da requisição, incluindo saltos para Redis, banco e fila — essencial ao extrair serviços.

● Health checks

`AspNetCore.HealthChecks` expõe prontidão e vivacidade de banco, Redis e broker para o orquestrador.

O coletor OpenTelemetry pode exportar para o backend de sua preferência — Grafana/Prometheus/Loki, Jaeger ou uma solução gerenciada — sem acoplar o código à ferramenta.

10 Escalabilidade e evolução

A escalabilidade tem duas faces, e a arquitetura responde às duas.

Escala de tráfego (runtime)

- **Leitura de agenda em cache:** a consulta mais repetida do sistema é servida pelo Redis, reduzindo pressão sobre o banco.
- **Escala horizontal do host:** a aplicação é stateless (sessão no Redis), então múltiplas instâncias atrás de um balanceador escalam de forma linear.
- **Trabalho assíncrono desacoplado:** lembretes, faturas e sincronização de convênios saem do caminho da requisição via fila.
- **Read replicas:** queries pesadas de relatório podem ir para réplicas de leitura do PostgreSQL.

Escala de organização e evolução para serviços

Quando um módulo (tipicamente a Agenda) precisar escalar isoladamente, a extração é direta porque a fronteira já existe:

Passo	Ação
1	O módulo já se comunica apenas por contratos e eventos — nenhuma dependência de código para quebrar.
2	Seu schema é separado; migra-se para um banco próprio sem desfazer <code>JOIN</code> cruzado.
3	Os eventos de domínio que já trafegam pela fila passam a cruzar a fronteira de rede sem mudança de contrato.
4	O serviço extraído sobe com deploy e escala independentes; o restante segue como monólito.

PRINCÍPIO

Não se paga o custo de um sistema distribuído antes de haver escala que o justifique. A arquitetura permite adiar essa decisão até que os dados de produção a exijam — e, quando exigirem, o caminho já está pavimentado.

11 Estrutura de solução

Organização física dos projetos na solução .NET. Cada módulo é um conjunto de projetos que espelha as quatro camadas.

```
OdontoPlatform.sln
├── src/
│   ├── Bootstrap/
│   │   └── OdontoPlatform.Api           # host único, composição de DI
│   ├── Modules/
│   │   ├── Scheduling/                # módulo Agenda
│   │   │   ├── Scheduling.Domain
│   │   │   ├── Scheduling.Application
│   │   │   ├── Scheduling.Infrastructure
│   │   │   └── Scheduling.Contracts    # API pública p/ outros módulos
│   │   ├── Records/                  # Prontuário
│   │   │   ├── Records.Domain
│   │   │   ├── Records.Application
│   │   │   ├── Records.Infrastructure
│   │   │   └── Records.Contracts
│   │   ├── Billing/                   # Financeiro
│   │   │   └── ... (mesma estrutura)
│   │   ├── Patients/                 # Pacientes / CRM
│   │   │   └── ... (mesma estrutura)
│   └── Shared/
│       ├── SharedKernel               # VOs, base de agregado, eventos
│       ├── Infrastructure.Common       # EF base, Outbox, tenancy
│       └── Contracts.Abstractions
├── tests/
│   ├── Scheduling.UnitTests
│   ├── Scheduling.IntegrationTests
│   └── ...
├── frontend/                          # SPA React + TypeScript
└── deploy/                            # Dockerfile, compose, manifests
```

FRONTEIRA APLICADA POR PROJETO

Um módulo só referencia o projeto `*.Contracts` de outro — nunca seu `Domain` ou `Infrastructure`. Essa regra transforma o acoplamento indevido em **erro de compilação**, não em algo que depende de disciplina do time.

12 DevOps e CI/CD

Etapa	Ferramenta / prática
Controle de versão	Git com trunk-based ou GitHub Flow; PRs revisados.
CI	GitHub Actions: restore, build, testes unitários e de integração (Testcontainers), análise estática.
Qualidade	Analyzers do .NET, cobertura mínima definida, <code>dotnet format</code> no pipeline.
Empacotamento	Imagem Docker multi-stage; frontend buildado e servido estaticamente ou via CDN.
Migrations	Aplicadas de forma controlada no deploy; nunca automaticamente em produção sem gate.
Ambientes	Dev, staging e produção com configuração por variáveis de ambiente / secrets.
Deploy	Rolling update no Kubernetes ou deploy em contêiner gerenciado; health checks como gate.

13 Roadmap de implementação

Ordem sugerida, entregando valor cedo e deixando a complexidade (convênios) para quando o núcleo estiver sólido.

Fase	Entrega	Foco técnico
Fase 1 Núcleo	Identity & Access, Tenancy, Pacientes e Agenda	Fundação multi-tenant, autenticação, CRUD e calendário.
Fase 2 Clínico	Prontuário eletrônico básico + anexos	Criptografia, auditoria, object storage.
Fase 3 Financeiro	Faturamento particular, depois convênios	Outbox, camada anticorrupção, jobs.
Fase 4 Inteligência	Relatórios, BI e dashboards	Projeções CQRS, read replicas.
Fase 5 Rede	Gestão de múltiplas unidades e estoque	Hierarquia de tenancy, isolamento por schema.

PONTO DE PARTIDA IMEDIATO

Comece pelo **SharedKernel** e pela **fundação de tenancy** (middleware + global query filter). Com essa base pronta, o módulo Agenda pode ser o primeiro vertical completo — do domínio à API — servindo de molde para todos os demais.

